Link de Comunicação SDR
Relatório das Etapas 1 e 2

SCM22108 Sistemas de Comunicação, Engenharia Eletrônica, IFSC

Gabriel de Oliveira Peres, matrícula 201920800001

04/11/2026

Versão do código: `bloco-02-rev2-sujo`

Repositório: Sistema_comunicacao_projeto_radio_digital

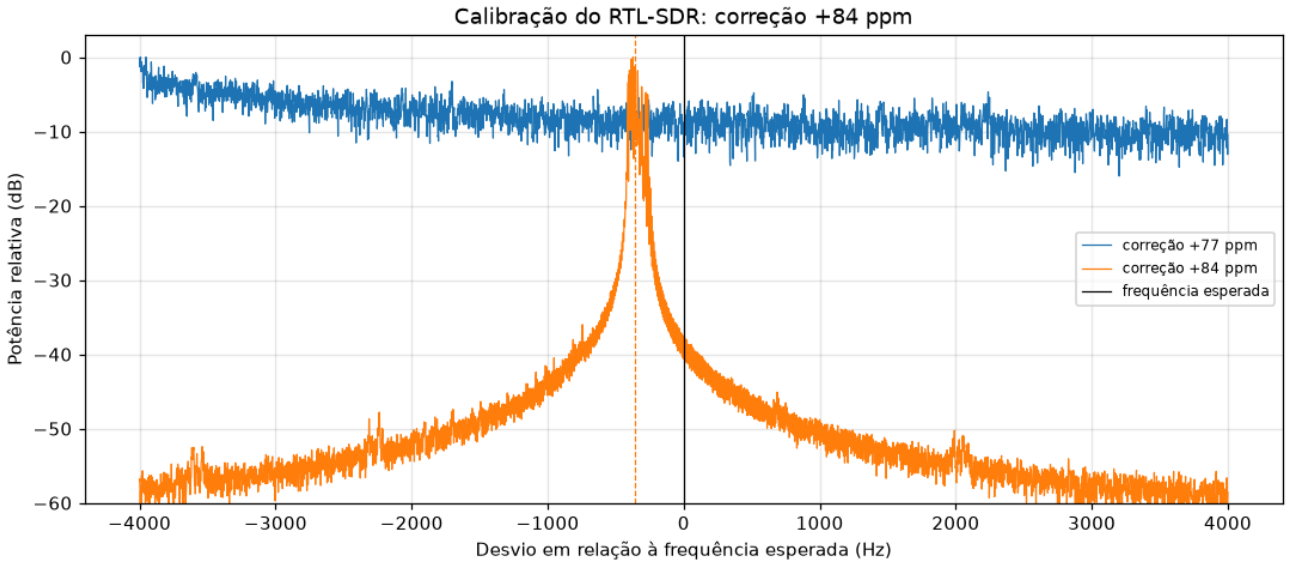
Sumário

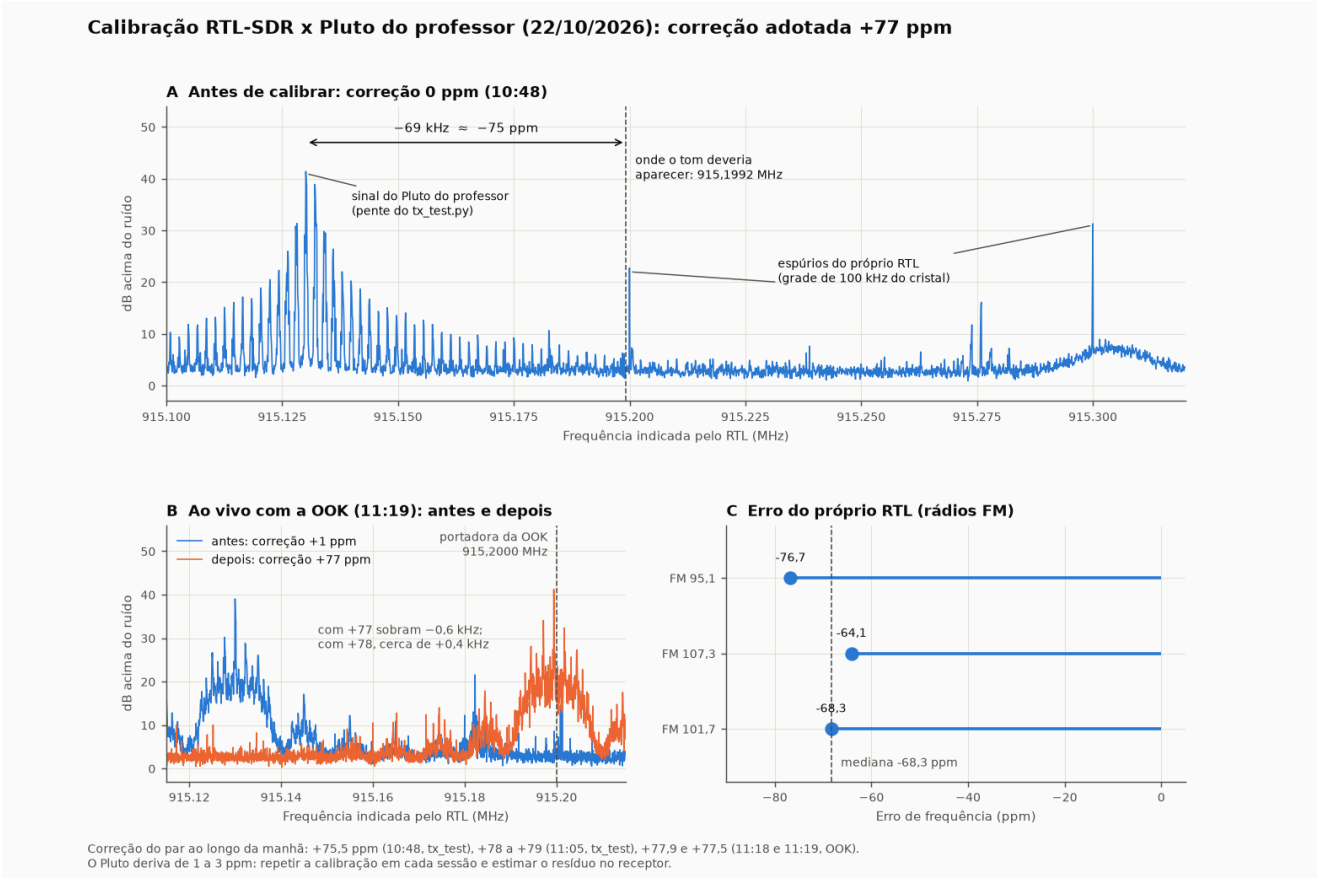
1. Calibração do RTL-SDR (Etapa 1)
2. Estudo e auditoria do código de referência (Etapa 2)
3. Recepção da OOK de referência no enlace real (Etapa 2)
4. Auditoria de conformidade das Etapas 1 e 2
5. Anexo A: registro de uso de IA

1. Calibração do RTL-SDR (Etapa 1)

Requisitos: **CAL-01** (correção de ppm calibrada e registrada) e **CAL-02** (offset residual medido).

> **Valor vigente (04/11/2026): +84 ppm**, para o par "Pluto da equipe + nosso RTL" (`configs/rtl.toml`). Medições de +84,04 e +84,21 ppm; resíduo logo após a calibração de −356 e −524 Hz. Detalhes na seção 10. O resíduo depois deriva: chegou a −1,6 kHz no fim da manhã (etapa2_recepcao_ook.md, seção 6). As seções 1 a 9 registram a revisão 2 (22/10, Pluto do professor, +77 ppm) e ficam como histórico.





Correção do par ao longo da manhã: +75,5 ppm (10:48, tx_test), +78 a +79 (11:05, tx_test), +77,9 e +77,5 (11:18 e 11:19, OOK).
O Pluto deriva de 1 a 3 ppm: repetir a calibração em cada sessão e estimar o resíduo no receptor.

1. Resultado da revisão 2 (22/10, Pluto do professor; histórico)

Item	Valor
Correção adotada	**+77 ppm**: decisão da equipe em 22/10/2026, para o par "nosso RTL + Pluto do professor" (`configs/rtl.toml`)
Correção do par medida ao longo da manhã	de +75,5 a +78,9 ppm; as duas últimas deram +77,9 e +77,5
Resíduo com +77 ppm	cerca de −0,6 kHz na OOK às 11:19; varia com a deriva do Pluto
Erro absoluto do nosso RTL (rádios FM)	de −64 a −77 ppm, mediana −68,3 ppm

O erro grande é principalmente do **nosso RTL**, apesar de o dongle ser vendido como "TCXO". O Pluto do professor contribui com os ppm restantes e **deriva de 1 a 3 ppm** ao longo de uma manhã. Por isso a calibração precisa ser repetida em cada sessão (depois dos 10 min de aquecimento), e o receptor ainda vai ter de estimar um resíduo de centenas de Hz, como o enunciado prevê.

2. Montagem

Item	Valor
Transmissor	ADALM-Pluto do professor: `tx_test.py` até ~11:06 e `pluto_tx_ook_v2.py` (OOK 10 kbit/s) a partir de ~11:18
Receptor	RTL-SDR RTL2832U + R820T2, Windows 11, driver WinUSB
Sintonia	914,95 MHz (o sinal esperado cai em +250 kHz, longe do pico DC)
Taxa / ganho	2,048 MS/s; 40 dB (40,2 dB aplicados) ou 30 dB, AGC desligado
Ferramentas	`scripts/calibrar_rtl.py`, `scripts/verificar_ppm_fm.py`, `scripts/figura_calibracao.py`

3. Onde cada transmissão do professor coloca a portadora

Programa	Portadora	Por quê
`tx_test.py`	**LO + 199,219 kHz** = 915,199219 MHz	O buffer cíclico tem 1024 amostras a 2 MS/s e o tom de 200 kHz completa 102,4 ciclos nele. O espectro vira um pente com passo de 1953,125 Hz, e o dente principal é o k = 102 (o k = 103 fica 3,5 dB abaixo). Achado A15.
`pluto_tx_ook_v2.py`	**LO + 200,000 kHz** = 915,200000 MHz	Cada bit tem 200 amostras e 20 ciclos inteiros da portadora, para qualquer mensagem.

Usar a referência errada desloca o resultado em 0,85 ppm. Por isso o script tem a opção `--tx-test`, que só deve ser usada quando o professor roda o `tx_test.py`.

4. Linha do tempo das medições

Horário	Sinal	Correção durante a captura	Correção total do par	Situação
10:48	`tx_test.py`	0	**+75,5 ppm**	reprocessado (`calibracao_20261022_111529.json`)
10:48		0	~~0 ppm~~	**inválido**: travou num espúrio do RTL (ver seção 5)
~11:05	`tx_test.py`	0, 1 e 77	+78,7 / +78,8 / +78,9 ppm	medições de diagnóstico (só impressas)
~11:06	`tx_test.py`	0, quatro sintonias	+77,9 ppm (média)	medição de diagnóstico; confirmou que o sinal é real
11:18	OOK	+1	**+77,9 ppm**	reprocessado com a referência da OOK (`..._112118.json`)

Horário	Sinal	Correção durante a captura	Correção total do par	Situação
11:19	OOK	+1, depois +77	**+77,5 ppm**; resíduo com +77 de cerca de −0,6 kHz	reprocessado (`..._112123.json`); figura, painel B

5. O que deu errado no caminho, e como foi corrigido

Ficam registrados, porque fazem parte da análise de falhas pedida no projeto.

1. **"Calibração de 0 ppm" (10:48).** A busca estreita, de ±10 kHz em volta de 915,2 MHz, travou numa raia que é **espúrio do próprio RTL**. Ele gera uma família de raias em múltiplos exatos de **100 kHz** (o cristal de 28,8 MHz dividido por 288) e de **256 kHz** (a taxa de amostragem dividida por 8). Como saem do próprio relógio do dongle, aparecem em números "redondos" mesmo com o RTL errando −68 ppm. **Correção:** o script descarta automaticamente raias nessa grade (`possivel_espurio_rtl`) e busca em ±150 kHz.
2. **Dente errado do pente do `tx_test.py`.** Uma estimativa intermediária usou o dente k = 101 no lugar do k = 102. **Correção:** a detecção usa o espectro médio, e os dentes vizinhos do mesmo pente não contam como sinais concorrentes (`mesmo_pente`).
3. **Referência errada quando o professor trocou para a OOK.** As rodadas ao vivo das 11:18 e 11:19 usaram `--tx-test` com o sinal já em OOK. Elas foram reprocessadas com a referência certa, e os registros originais foram mantidos com uma nota.
4. **Captura de "Pluto desligado" contaminada.** O professor voltou a transmitir durante a captura de fundo das 11:16. Ela foi descartada e não é usada como evidência.

6. Por que o `rx_test_915.py` deu 77 e como usar o 77

- O script do professor procura o maior pico da banda inteira com FFT de 500 Hz por bin (achado A6). Neste RTL isso funciona só quando o sinal do Pluto é o mais forte da banda.
- Ele descarta a correção inicial (achado A5). Partindo de 1 ppm, ele mediu 77 e aplicou 77. Pela própria medição dele, o total seria 78. A diferença entre 77 e 78 fica dentro da deriva do Pluto, e a equipe adotou **77**.
- **Atenção:** colocar `freq_correction = 77` no início do `rx_test_915.py` **não** mantém os 77. O script mede o que sobra (perto de 0), aplica só isso e descarta os 77. A correção deve ser fixada no **receptor**: `FREQ_CORRECTION_PPM = 77` no `rtl_rx_ook_v2.py` (Bloco 2) e `configs/rtl.toml` no nosso código.

7. Impacto nas próximas etapas

- **OOK:** o resíduo de centenas de Hz não afeta (filtro de ±10 kHz, detecção de envoltória).
- **BPSK coerente:** com −0,6 kHz de resíduo, a fase gira 21,6° por símbolo a 10 kbaud e cerca de 2,2° a 100 kbaud. Num pacote de 1109 símbolos isso dá várias voltas. Então a BPSK precisa de DBPSK ou de estimar o resíduo pelo preâmbulo (decisão D2). A deriva do Pluto, de centenas de Hz por minuto, reforça essa necessidade.
- **Faixa ISM:** há outros sinais na banda, e o nosso RTL tem espúrios próprios em 915,0, 915,2 e 915,3 MHz. A escolha do deslocamento do nosso sinal deve evitar essas frequências.

8. Como reproduzir

Com o RTL ligado há pelo menos 10 min:

```
# professor rodando o tx_test.py
.venv\Scripts\python scripts\calibrar_rtl.py --freq-tom-hz 915.2e6 --tx-test --correcao-inicial-ppm 1 --salvar-iq
# professor rodando a OOK (pluto_tx_ook_v2.py)
.venv\Scripts\python scripts\calibrar_rtl.py --freq-tom-hz 915.2e6 --correcao-inicial-ppm 1 --salvar-iq
# erro absoluto do RTL, sem depender do Pluto
.venv\Scripts\python scripts\verificar_ppm_fm.py
# figura-resumo a partir dos arquivos gravados
.venv\Scripts\python scripts\figura_calibracao.py
```

9. Evidências (`dados/calibracao/`)

Arquivo	Conteúdo
`calibracao_20261022_104830.*`	captura das 10:48 (`tx_test.py`); o JSON original está marcado como inválido
`calibracao_20261022_111529.json/.png`	reprocessamento das 10:48: +75,5 ppm
`calibracao_20261022_111837.*`, `..._111918.*`	rodadas ao vivo (OOK), com nota de referência incorreta
`calibracao_20261022_112118.json`, `..._112123.json`	reprocessamentos com a referência da OOK: +77,9 e +77,5 ppm
`fm_20261022_111611.json`	erro absoluto do RTL pelas emissoras FM

As capturas IQ (`.npy`) ficam só no disco, fora do git.

---

10. Revisão 3: calibração com o Pluto da equipe (04/11/2026)

A calibração de 22/10 foi feita com o Pluto do professor. Em 04/11 o par foi medido com o **Pluto da equipe**, o que de fato vai transmitir no projeto: ADALM-Pluto Rev.C (AD9364), firmware v0.32, IP 192.168.2.22, serial terminando em `bc7b7b4d52`, `xo_correction` de fábrica = 39 999 784 Hz.

Transmissor próprio: `scripts/transmitir_tom.py` (`sdrlink/hw/pluto_tx.py`). O tom fecha ciclos inteiros no buffer, sem o pente do `tx_test.py`. A duração é fixa e o buffer é sempre desligado no final. Usei o tom em **915,25 MHz** (LO 915,0 MHz + 250 kHz), fora das grades de espúrios do RTL, com ganho de TX de −30 dB e o RTL em 20 dB, porque os dois rádios estão lado a lado.

Horário	Medida	Resultado
09:57	Rádios FM, RTL recém-conectado, com +77 ppm	erro do RTL de −76,9 ppm (mediana); ainda frio
10:00	Tom do Pluto, RTL com cerca de 5 min	tom 64,7 dB acima do ruído; par −83,6 ppm (teste, RTL ainda frio)
10:01	Controle com o Pluto **desligado**	o tom some (9 dB, só ruído). Fica outro sinal ISM em 915,2432 MHz, a 750 Hz do tom, com ~18 dB
10:06	**Tom, RTL aquecido**	**par −84,04 ppm, correção +84, resíduo medido −356 Hz**
10:09	**Tom, RTL aquecido**	**par −84,21 ppm, correção +84, resíduo medido −524 Hz**
10:10	Rádios FM, RTL aquecido	erro do RTL de −74,6 ppm (mediana)

Conclusões: - O par **Pluto da equipe + RTL** precisa de **+84 ppm**. O valor de 77 ppm vale para o Pluto do professor: cada Pluto tem o seu próprio erro de oscilador. - Pelas rádios FM, o RTL sozinho erra cerca de −75 ppm depois de aquecido, e mais 2 a 3 ppm quando ainda está frio. Logo, o Pluto da equipe erra cerca de −9,5 ppm. Ele pode ser corrigido pelo `xo_correction` (requisito CAL-03), se a equipe quiser deixar a correção do RTL igual para qualquer Pluto. - Mesmo com a correção, sobra um resíduo de centenas de Hz que deriva cerca de 1 Hz/s logo depois do aquecimento. O receptor precisa estimar esse resíduo. - Medição posterior, durante a

Etapa 2: o resíduo foi de −0,36 kHz (10:06) a −1,6 kHz (10:42), cerca de 1,4 ppm em 35 min, e oscila uns 400 Hz dentro de uma transmissão de 60 s (achado A18, etapa2_recepcao_ook.md).

Falha encontrada e corrigida: a primeira tentativa abortou porque o Pluto sintonizou o LO 2 Hz abaixo do pedido (resolução do sintetizador), e a checagem aceitava só 1 Hz. Sem o tom no ar, a calibração travou no sinal ISM de 915,2432 MHz e deu um resultado falso; esse registro foi apagado. A tolerância do LO passou a 10 Hz. Lição: **sempre fazer a captura de controle com o Pluto desligado**.

Evidências: `dados/calibracao/calibracao_20261104_100033`, `..._100631` e `..._100917` (JSON e figura), `fm_20261104_095704.json` e `fm_20261104_101037.json`.

2. Estudo e auditoria do código de referência (Etapa 2)

4. Auditoria do código de referência (base da Etapa 2)

Rodei a simulação do professor (`tx_ook_sim.py` → `canal_sim.py` → `rx_ook_sim.py`) com semente fixa: a mensagem `HELLO WORLD MANDRIAUM` foi decodificada com **CRC OK e 0 de 184 bits errados**, com SNR de 10 dB e offset de +1,5 kHz. A referência funciona na condição para a qual foi feita. Os pontos abaixo são o que **não atende** ao projeto final ou precisa de cuidado.

4.1 Mapa das funções

Arquivo	Papel	Funções principais
`tx_test.py`	Pluto transmite um tom em +200 kHz	(script linear)
`rx_test_915.py`	RTL mede o tom e estima o erro em ppm	`capture_chunks`, `find_peak`
`Tx_Rx_OOK/pluto_tx_ook_v2.py`	TX OOK no Pluto, quadro repetido em buffer cíclico	`crc8`, `build_frame_bits`, `build_ook_envelope`
`Tx_Rx_OOK/rtl_rx_ook_v2.py`	RX OOK: translação, filtro, envoltória, Barker, limiar, CRC	`capture_chunks`, `simple_lowpass`, `crc8`, `bits_to_levels`, `read_bits`
`OOK_simulacao/*`	Mesma cadeia, com `.npy` no lugar do hardware e gráficos passo a passo	idem + `field_boundaries`, `bit_means`, `psd_db`
`teste_FM.py`	Varredura de espectro FM 88 a 108 MHz (teste do RTL)	`capture_spectrum`
`instalar.sh`	Instala ambiente Python do RTL no Fedora	(shell)

Quadro de referência: `[preâmbulo 16][Barker-13][comprimento 8][payload 8·N][CRC-8][guarda 16]`, OOK 10 kbit/s, rampas de cosseno levantado (10% do bit), tom em +200 kHz, TX 2,0 MS/s, RX 2,048 MS/s.

4.2 Achados

#	Gravidade	Onde	Achado	Consequência para o projeto	Requisito
A1	**Crítico**	`rtl_rx_ook_v2.py`	O RX decodifica **um único quadro** por captura (`argmax` da correlação)	Impossível medir PER e goodput, que exigem decodificar todos os pacotes de um ensaio	MET-01, MET-02
A2	**Crítico**	RX	Não há **limiar de detecção**: o maior pico da correlação sempre vira "quadro", mesmo só com ruído	Falsos quadros; precisa de detecção com limiar e validação	RX-01
A3	**Crítico**	quadro	Sem **número de sequência**; CRC-8 com `init = 0x00`	Não identifica pacotes nem duplicatas; CRC-8 detecta mal erros em 128 bytes	PKT-01, PKT-02
A4	**Crítico**	`pluto_tx_ook_v2.py`	TX repete **o mesmo quadro** em buffer cíclico, sem log	Na prova, cada pacote tem payload diferente vindo de arquivo, e o TX precisa registrar o que transmitiu	PKT-04, TX-01
A5	Alto	`rx_test_915.py`	Começa com `freq_correction = 34` e depois aplica **só o erro medido**, descartando os 34 ppm já aplicados	A correção final fica errada, a menos que a calibração parta de 0 ppm. Validar no hardware	CAL-01

#	Gravidade	Onde	Achado	Consequência para o projeto	Requisito
A6	Alto	`rx_test_915.py`	Resolução da FFT: 2,048 MHz / 4096 = **500 Hz por bin**	Não mede um residual de "centenas de Hz", que é exatamente o que o professor pede. Precisa de FFT maior ou interpolação	CAL-02
A7	Alto	`rx_test_915.py`	O erro medido mistura o erro do Pluto e o do RTL (mede um contra o outro)	Precisa ser declarado no registro de calibração; ajustar também o Pluto (`xo_correction`)	CAL-01, CAL-03
A8	Alto	RX	`read_samples` devolve `complex128`	60 s a 2,4 MS/s = **2,3 GB** de RAM. Para a prova, gravar os bytes brutos (`uint8`, 288 MB)	RX-02
A9	Médio	RX	`MAX_EXPECTED_PAYLOAD_BYTES = 64`	Menor que os 128 bytes da prova	PKT-04
A10	Médio	RX	Amostras por bit pela taxa nominal (204,8), sem recuperação de relógio	Funciona em quadros curtos; em taxas altas o desvio de relógio acumula. O próprio PDF pede que isso seja entendido	Etapa 1
A11	Médio	`instalar.sh`	Só funciona no Fedora; o passo [1/5] não instala nada; **não instala pyadi-iio nem libiio**; `scipy` aparece duas vezes; não trata o driver DVB do kernel	O PC do TX não fica pronto com esse script	PRC-01
A12	Baixo	`teste_FM.py`	Força o backend `Qt5Agg`, que exige PyQt5 (não instalado pelo script)	Pode falhar ao abrir o gráfico	(teste)
A13	Baixo	TX	`PLUTO_URI = "ip:192.168.2.22"`; o padrão de fábrica é `192.168.2.1`	Confirmar o IP do Pluto da bancada	PLT-01
A14	Baixo	todos	Parâmetros espalhados como constantes em cada script, sem arquivo de configuração	O requisito pede parâmetros configuráveis e o TX e o RX precisam concordar	PRC-01
A15	Alto	`tx_test.py`	O tom de 200 kHz completa 102,4 ciclos no buffer cíclico de 1024 amostras: o espectro vira um pente de 1953 Hz e a raia principal fica em LO + 199,219 kHz	Calibrar contra 915,2 MHz exatos erra 0,85 ppm (descoberto no ensaio de 22/10)	CAL-01
A16	Alto	RTL-SDR do projeto	O dongle erra cerca de −68 ppm (medido pelas rádios FM), apesar de vendido como TCXO, e gera espúrios em múltiplos de 100 kHz e de taxa/8	Uma calibração que trave num espúrio dá um resultado falso (aconteceu em 22/10)	CAL-01, CAL-02
A17	Alto	`rtl_rx_ook_v2.py` (`capture_chunks`) e todo leitor síncrono do RTL	A leitura síncrona em blocos de 256 k amostras perde de 2 a 5 mil amostras em **cada emenda** (a cada 128 ms), cerca de 1,3% do sinal	Quadros que cruzam a emenda falham no CRC ou somem (12 de 107 no ensaio de 04/11); PER e goodput sairiam errados. Corrigido com leitura assíncrona (etapa2_recepcao_ook.md)	RX-02, MET-01
A18	Alto	enlace Pluto da equipe → RTL	Mesmo com +84 ppm, o offset residual não é fixo: foi de −0,36 a −1,62 kHz em 35 min e andou 440 Hz dentro de uma transmissão de 60 s (até cerca de −35 Hz/s); a temperatura do chip do Pluto ficou parada	Na BPSK, 2 kHz giram a fase 20 voltas por pacote de 10 ms: o receptor tem de estimar o offset em cada pacote, com busca de pelo menos ±3 kHz	CAL-02, MOD-03

#	Gravidade	Onde	Achado	Consequência para o projeto	Requisito
A19	Baixo	`sdrlink.hw.rtl_rx` (leitura assíncrona)	Em cerca de 5% das capturas a librtlsdr acusa erro de I2C ao pôr o sintonizador em repouso no fechamento; nunca com leitura síncrona	Não afeta os dados (captura sempre completa e a seguinte funciona); fica sob observação	RX-02
A20	Médio	ensaios de 04/11 e referência (`CENTER_FREQ = 915e6`)	O LO do Pluto ficou em 915,000 MHz, exatamente na borda da faixa autorizada; o vazamento do LO fica na borda e a imagem IQ (LO − 200 kHz = 914,8 MHz) cai fora dela. O sinal útil (915,2 MHz) estava dentro	Nos próximos ensaios o LO vai para perto do meio da faixa; o código do projeto recusa transmitir se sinal, LO ou imagem saírem de 915 a 928 MHz	PLT-05

Conclusão da auditoria: a referência é uma **boa base didática** (quadro, Barker, limiar pelo preâmbulo, CRC), mas atende só MOD-01 e parte de PKT-01. O restante tem de ser construído. Os achados A1 a A4 são o núcleo do trabalho de engenharia.

5. Cálculos que orientam o projeto

Todos os números abaixo foram calculados (script de verificação em Python) e serão revalidados na simulação.

5.1 Overhead do quadro (payload de 128 bytes = 1024 bits)

Bits por quadro = preâmbulo 16 + sync 13 + comprimento 8 + sequência + 1024 + CRC + guarda 16.

Formato	Bits/quadro	Eficiência	Goodput a 10 kbit/s	Taxa mínima para 10 kbit/s com PER 1%
Referência (sem sequência, CRC-8)	1085	94,4%	9,44 kbit/s	10,70 kbit/s
+ sequência 16, CRC-8	1101	93,0%	9,30 kbit/s	10,86 kbit/s
+ sequência 16, **CRC-16**	1109	92,3%	9,23 kbit/s	**10,94 kbit/s**
+ sequência 16, CRC-32	1125	91,0%	9,10 kbit/s	11,10 kbit/s

Isso confirma a observação do PDF: a OOK de referência a 10 kbit/s **não passa** na prova B. Para a prova B com folga: taxa de pelo menos **16 a 20 kbit/s**.

5.2 Banda ocupada a 99% (limite da prova: 1 MHz)

Pulso	Banda a 99%	Taxa de símbolos máxima em 1 MHz
Retangular (BPSK/OOK)	≈ 20 × Rs	≈ 50 kbaud
RRC, α = 0,25	1,10 × Rs	≈ 900 kbaud
RRC, α = 0,35	1,17 × Rs	≈ 850 kbaud
RRC, α = 0,50	1,26 × Rs	≈ 790 kbaud
2-FSK (Carson, aproximado)	≈ 2(Δf + Rb)	depende do desvio Δf

Conclusão: **pulso retangular inviabiliza a prova A**; a formatação RRC (ou cosseno levantado) é o que libera taxa. O limite prático passa a ser o RTL-SDR: a 2,4 MS/s a janela é ±1,2 MHz e o sinal deve ficar longe do DC (pico do RTL) e das bordas do filtro anti-aliasing.

5.3 Offset residual e BPSK

Depois da calibração sobra um offset Δf de centenas de Hz (1 ppm em 915 MHz = 915 Hz). A fase gira 360°·Δf/Rs a cada símbolo:

Δf	Rs = 10 kbaud	Rs = 100 kbaud	Rs = 250 kbaud
100 Hz	3,6°/símbolo, 3992° no pacote	0,36°, 399°	0,14°, 160°
300 Hz	10,8°/símbolo, 11977° no pacote	1,08°, 1198°	0,43°, 479°
500 Hz	18°/símbolo, 19962° no pacote	1,8°, 1996°	0,72°, 798°

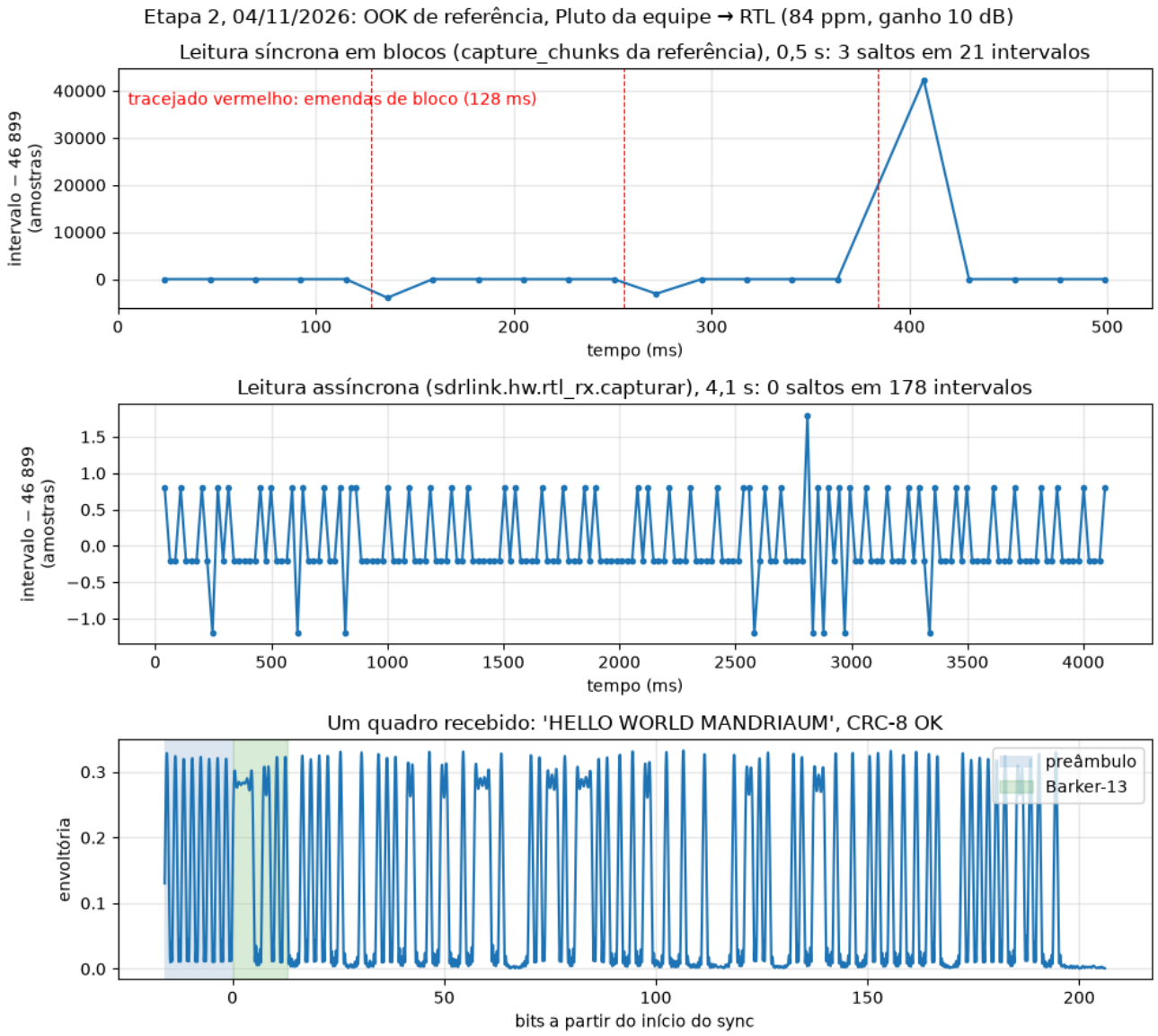
Conclusão: BPSK coerente **sem** tratar o offset não funciona (a fase dá várias voltas dentro de um pacote). Opções: (a) **DBPSK**, que só depende da diferença de fase entre símbolos vizinhos (de 0,1° a 18° por símbolo, tolerável); (b) estimar o offset pelo preâmbulo e corrigir, mais uma malha de fase. O PDF exige, se usarmos DBPSK, a comparação com BPSK coerente em simulação.

5.4 Calibração

- Resolução de frequência da FFT = fs / N. Com 2,048 MS/s e N = 4096, são 500 Hz. Para medir o residual com resolução de cerca de 10 Hz, usar $N \geq 2^{20}$ (cerca de 2 Hz) ou interpolar o pico.
- ppm = (f_medida − f_esperada) / f_portadora × 10^6, aplicada **sobre** a correção já ativa (achado A5).

3. Recepção da OOK de referência no enlace real (Etapa 2)

Requisitos: **MOD-01** (OOK no enlace real), **RX-02** (captura gravada), **CAL-01/CAL-02** (correção aplicada e offset residual medido). Achados novos: **A17**, **A18** e **A19**.



1. Resultado

Item	Valor	Evidência
Mensagem recebida	`HELLO WORLD MANDRIAUM` (21 bytes), CRC-8 OK	logs abaixo
Receptor da referência, sem alteração de lógica	**8 de 8** recepções com CRC OK (5 às 10:16 e 3 às 10:38)	`dados/logs/etapa2_rx_*.log`, seção 3
Todos os quadros das 5 capturas da referência (leitura síncrona, 0,5 s cada)	95 com CRC OK, **12 com CRC falho**, 2 não detectados	seção 5
Todos os quadros da captura contínua de 16 s (leitura assíncrona)	**698 de 698 com CRC OK**	seção 4
PER medida na captura de 16 s	**0 / 698 = 0%** (limite da prova: 1%)	seção 4

Item	Valor	Evidência
BER do payload na captura de 16 s	**0 / 117 264 bits**	seção 4
Offset residual da portadora	de **−0,68 a −1,62 kHz** ao longo da manhã, com +84 ppm aplicados	seção 6
Ganho do RTL	pedido 10 dB, **aplicado 8,7 dB** (degrau do R820T), AGC desligado	`Captura.ganho_db`
Correção de frequência	+84 ppm (calibração de 04/11)	

A OOK de referência funciona no enlace real. As falhas de CRC que apareceram não vêm do canal: vêm da forma como a referência lê o RTL, que perde amostras a cada 128 ms (achado A17, seção 5). Com a leitura assíncrona do nosso pacote nenhuma amostra se perde e todos os quadros passam no CRC.

Sobre PER e BER. O limite das provas é **PER ≤ 1%** (seções 8 e 12 do enunciado); a BER não tem limite, só precisa ser medida (Etapa 5). Os 1,3% citados na seção 5 são a fração de *amostras* perdida pela leitura da referência, não uma BER. Neste ensaio o quadro é sempre o mesmo e não tem número de sequência, então a PER aqui é contada pelo período fixo do TX; a PER oficial, pelo log do transmissor, só existe a partir do quadro com número de sequência da Etapa 3.

2. Montagem

Item	Valor
Transmissor	Pluto da equipe (Rev.C AD9364, fw v0.32, `ip:192.168.2.22`), `pluto_tx_ook_v2.py`, ganho de TX −20 dB (padrão da referência)
Sinal	OOK 10 kbit/s, portadora em LO + 200 kHz = 915,200 MHz, quadro repetido em buffer cíclico. A faixa autorizada (915 a 928 MHz) foi definida pelo professor depois destes ensaios: o sinal estava dentro, mas o LO (915,0 MHz) ficou na borda e a imagem IQ (914,8 MHz) fora (achado A20). Os próximos ensaios usam o LO perto do meio da faixa
Receptor	RTL-SDR RTL2832U + R820T2, 915 MHz, 2,048 MS/s, AGC desligado
Distância	rádios lado a lado na bancada
Computadores	**um só PC** para TX e RX neste ensaio (ver pendência P2 em auditoria_etapas_1_2.md)
Aquecimento	RTL ligado desde antes da primeira medição FM (09:57), mais de 15 min antes do ensaio

3. O que mudou nas cópias de ensaio e o log da recepção

Os programas do professor ficam intactos em `referencia/`. O ensaio usa cópias geradas por `ensaios/etapa2/gerar_copias.py`, e o diff completo está em `ensaios/etapa2/diferencas.diff`.

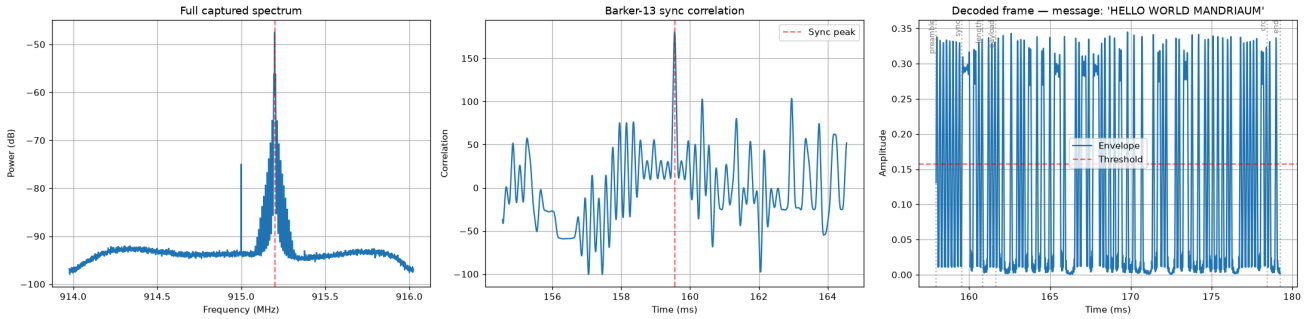
Arquivo	Mudança	Motivo
TX	`while True` trocado por 60 s fixos, com `tx_destroy_buffer` no `finally`	o ensaio roda sem teclado e o Pluto não pode ficar transmitindo
RX	`GAIN = 30` para `GAIN = 10`	saturação (seção 4.1)
RX	`FREQ_CORRECTION_PPM = 34` para `84`	calibração do par com o Pluto da equipe
RX	grava a captura (`.npy`) e a figura (`.png`) em `dados/capturas/`	evidência da recepção

A demodulação, o sincronismo, o limiar e o CRC da referência não foram tocados.

Log de console de uma recepção (`dados/logs/etapa2_rx_20261104_103811.log`, as outras duas das 10:38 são iguais em forma):

```
Capturing ~0.50 s at 915.000 MHz, gain=10 dB, ppm=84...
  chunk 1/4 captured  chunk 2/4 captured  chunk 3/4 captured  chunk 4/4 captured
Total samples: 1048576 (~512.0 ms)
Sync correlation peak at sample 326763 (t=159.55 ms), strength=180.52
Preamble levels: high=0.2639, low=0.0502, threshold=0.1571
Decoded length byte: 00010101 = 21 bytes
Decoded message: 'HELLO WORLD MANDRIAUM'
CRC-8 received=0x44 computed=0x44 -> OK ✅
```

Figura gerada pelo próprio receptor da referência nessa recepção:



4. Captura contínua: PER, BER e saturação

Captura de 16,0 s às 10:38:41 com `sdrlink.hw.rtl_rx.capturar` (leitura assíncrona), processada por `decodificar_quadros_ensaio.py`, que usa as funções `crc8`, `bits_to_levels` e `read_bits` da própria referência:

```
16.00 s, 698 quadros achados de 698 esperados, 0 saltos de intervalo
{'CRC OK': 698} não detectados: 0
{'HELLO WORLD MANDRIAUM': 698}
PER = 0/698 = 0.00%
BER = 0/117264 bits = 0.00e+00
offset residual da portadora: -1343.0 Hz
```

Política de contagem (seção 7 do enunciado pede que seja declarada): quadros esperados pelo período fixo do TX, do primeiro ao último sincronismo; não detectados e CRC falho contam como falha na PER; o quadro cortado pelo fim da janela de captura não entra (limite da captura, não do enlace). A BER compara o payload decodificado com a mensagem conhecida em todos os quadros detectados com o comprimento certo, inclusive os de CRC falho; os quadros não detectados ficam fora da BER e aparecem na PER.

4.1 Saturação com o ganho da referência

Com os rádios lado a lado, o Pluto em −20 dB e o RTL em 30 dB, de 26% a 29% das amostras ficaram no fundo de escala do ADC de 8 bits. A envoltória fica achatada e uma das recepções falhou no CRC. Testei os ganhos com o mesmo sinal:

Ganho do RTL	Amostras saturadas	Pico da amplitude
30 dB	26% a 29%	fundo de escala
20 dB	0%	0,79 do fundo de escala
10 dB (8,7 aplicado)	0%	0,32 do fundo de escala

Adotei 10 dB por margem. O ganho certo depende da distância, por isso `sdrlink.hw.rtl_rx.fracao_saturada` passa a ser conferida em todo ensaio; acima de 0,1% o ganho tem de baixar. A captura de 16 s teve 0% de saturação.

5. Achado A17: a leitura em blocos perde amostras

5.1 Como apareceu

A referência decodifica só o primeiro quadro de cada captura (A1). Decodificando **todos** os quadros de cada captura, de 2 a 3 por captura falhavam no CRC, sempre nos mesmos pontos no tempo.

O transmissor repete o quadro a cada 229 bits × 204,8 amostras = **46 899,2 amostras**, sem variação. O intervalo medido entre os sincronismos bate com esse valor em quase todo lugar, menos exatamente nas emendas dos blocos de 256 k amostras (128 ms) do `capture_chunks`:

Captura (04/11)	Amostras perdidas em cada emenda
10:16:54	3 948, 3 102, 4 512 (nesta o quadro inteiro sumiu)
10:17:00	2 727, 3 008, 3 477
10:17:05	4 324, 5 075, 3 665
10:17:10	2 256, 2 350, 2 632
10:17:16	3 760 (quadro sumiu), 3 008, 2 820

São 15 perdas em 15 emendas, de 1,1 a 2,5 ms cada, cerca de 1,3% das amostras. Um quadro que atravessa a emenda fica com um buraco no meio e falha no CRC; se o buraco cai no sincronismo, o quadro nem é detectado. Na prática, com o leitor da referência, a PER ficaria perto de 13% (14 falhas em 109 quadros).

5.2 Causa

`read_samples` e `read_bytes` fazem uma transferência USB síncrona por chamada. Entre uma chamada e a próxima (conversão para complexo e cópia no Python), o RTL continua amostrando e o buffer interno transborda. Nossa primeira versão de `capturar` lia do mesmo jeito e perdeu 31 vezes em 4,1 s.

5.3 Correção

`sdrlink.hw.rtl_rx.capturar` passou a usar a **leitura assíncrona** da librtlsdr (`read_bytes_async`). A biblioteca mantém 15 buffers USB em fila e chama um callback a cada bloco. O callback (`Acumulador`) só copia o bloco para um vetor pré-alocado e cancela a leitura quando o vetor enche. Um temporizador garante o término mesmo se o rádio parar de responder (regra 2 do padrão de código).

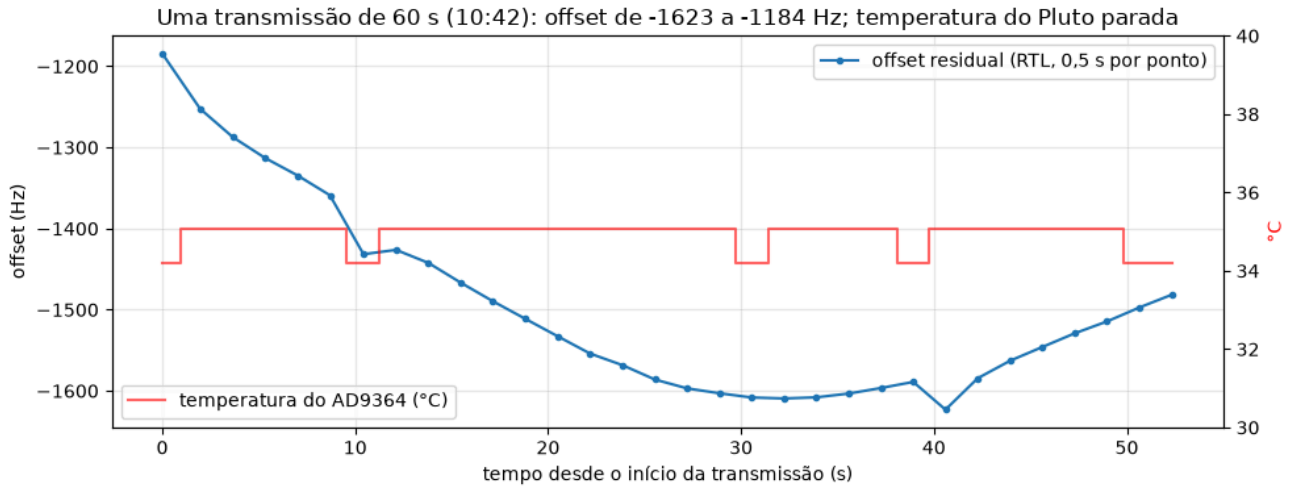
Ao cancelar, a librtlsdr devolve o código do último `libusb_cancel_transfer`, que é `LIBUSB_ERROR_NOT_FOUND` quando a transferência já tinha terminado. O pyrtlsdr transforma isso em exceção e fecha o rádio. Com o vetor cheio esse erro é o fim normal da leitura e é ignorado; com o vetor incompleto vira `RuntimeError`. Os dois casos têm teste automático.

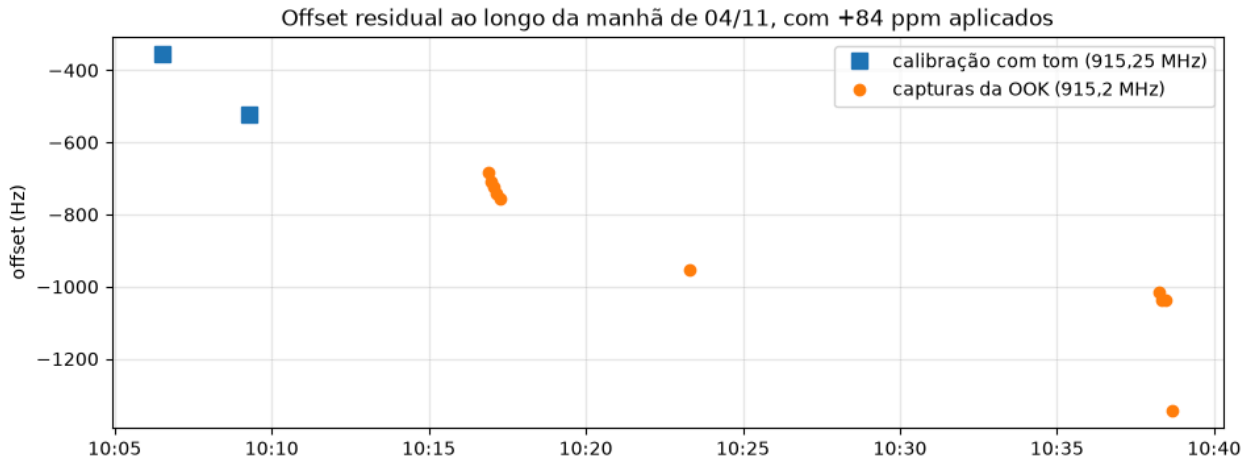
5.4 Verificação

Captura	Leitura	Saltos de intervalo	Desvio máximo do intervalo	CRC OK
5 capturas da referência, 0,5 s	síncrona	15	5 075 amostras	95 de 109
`capturar`, 4,1 s (10:23)	assíncrona	**0**	**1,8 amostra**	**178 de 178**
`capturar`, 16,0 s (10:38)	assíncrona	**0**		**698 de 698**

O desvio de ±2 amostras que sobra é só o arredondamento do pico da correlação.

6. Achado A18: o offset residual deriva





O enunciado pede medir o offset residual e explicar como lidamos com ele. Medi a posição da raia da portadora da OOK (FFT longa com interpolação, função `offset_residual_hz`) em todas as capturas e num ensaio dedicado (`deriva_offset_ensaio.py`): uma transmissão de 60 s com uma medida de 0,5 s a cada 1,7 s, lendo junto a temperatura do AD9364 pelo sensor interno do Pluto.

Momento	Offset residual
10:06 e 10:09 (calibração com tom)	−356 e −524 Hz
10:16 a 10:17 (5 capturas)	−685 a −758 Hz
10:23	−953 Hz
10:38	−1 015 a −1 343 Hz
10:42, dentro de uma transmissão de 60 s	de −1 184 a −1 623 Hz (registro `dados/logs/deriva_offset_20261104_104204.json`)

O que isso mostra:

1. **Deriva lenta**: cerca de −1 kHz (1,1 ppm) em 35 minutos, sempre no mesmo sentido; somando a variação rápida, o offset foi de −0,36 a −1,62 kHz.
2. **Variação rápida**: dentro de uma transmissão de 60 s o offset andou 440 Hz, com inclinação de até cerca de −35 Hz/s nos primeiros segundos.
3. **A temperatura do chip do Pluto ficou parada** em 34 a 35 °C (resolução de 1 °C), então ela não explica a variação. Com um par de rádios só não dá para separar a parte do Pluto da parte do RTL; isso fica para um ensaio com as rádios FM medidas no mesmo intervalo (pendência P4 da auditoria).

Como lidamos com ele:

- **OOK**: o detector de envoltória não depende da fase, e o filtro de ±10 kHz da referência tem folga para ±1,6 kHz. Nada a fazer.
- **2-FSK e BPSK (Etapa 3)**: o receptor **não pode supor offset fixo**. Um pacote de 128 bytes dura cerca de 10 ms a 100 kbit/s; nesse tempo a deriva (até cerca de 35 Hz/s) muda o offset em menos de 1 Hz, mas o valor absoluto, de até cerca de 2 kHz, gira a fase da BPSK em 20 voltas a cada 10 ms. Por isso o offset será **estimado em cada pacote** a partir do preâmbulo (ou será usada DBPSK, com a comparação em simulação que o enunciado exige), e a faixa de busca será de pelo menos ±3 kHz.
- **Calibração**: repetir no início de cada sessão de bancada, depois do aquecimento, e registrar o offset junto com cada ensaio.

7. Achado A19: erro de I2C ao fechar o RTL depois da leitura assíncrona

Em algumas capturas a librtlsdr imprime, ao fechar o rádio:

```
r82xx_write_arr: i2c wr failed=-6 reg=06 len=1
rtlsdr_demod_write_reg failed with -6
```

Teste (04/11)	Ocorrências
12 capturas seguidas com `capturar` (assíncrona, pyrtlsdr)	2
20 capturas com a chamada direta à librtlsdr, sem o fechamento antecipado do pyrtlsdr	1
25 capturas com a chamada direta e pausa de 100 ms antes de fechar	1
20 capturas com leitura síncrona	0

O erro acontece na escrita que põe o sintonizador em repouso, **depois** que a captura já terminou. Em todas as ocorrências a captura estava completa (o código confere) e a captura seguinte funcionou: foram mais de 70 aberturas seguidas sem falha. Conclusão: não afeta os dados; fica registrado e sob observação. Se aparecer falha na abertura seguinte, a alternativa é um leitor síncrono em thread separada com fila, que também não perde amostras.

8. Linha do tempo

Horário	O quê	Resultado
antes de 10:16	Primeiras recepções com ganho 30 dB (capturas apagadas)	26% a 29% saturado, uma falha de CRC
10:16 a 10:17	5 recepções com a cópia de ensaio (ganho 10 dB, 84 ppm)	5 de 5 com CRC OK
depois	Decodificação de todos os quadros das 5 capturas	12 falhas, todas nas emendas: achado A17
10:23	Captura contínua de 4,1 s com a leitura assíncrona, duas rodadas	0 saltos, 178 de 178 com CRC OK
10:38	3 recepções da referência com log de console	3 de 3 com CRC OK
10:38	Captura contínua de 16 s	698 de 698, PER 0%, BER 0
10:42	Ensaio de deriva durante uma transmissão de 60 s	offset de −1,18 a −1,62 kHz: achado A18
10:45 em diante	Testes do erro de I2C	achado A19

9. Como reproduzir

```
python ensaios/etapa2/gerar_copias.py
python ensaios/etapa2/pluto_tx_ook_v2_ensaio.py        # PC do TX, 60 s
python ensaios/etapa2/rtl_rx_ook_v2_ensaio.py          # PC do RX, grava em dados/capturas/
python ensaios/etapa2/decodificar_quadros_ensaio.py dados/capturas/<captura>.npy
python ensaios/etapa2/deriva_offset_ensaio.py 54       # com o TX ligado; grava em dados/logs/
python ensaios/etapa2/figura_etapa2_ensaio.py <captura_ref.npy> <captura_continua.npy>
docs/figuras/etapa2_04nov.png
python ensaios/etapa2/figura_deriva_ensaio.py dados/logs/deriva_offset_<...>.json
docs/figuras/deriva_offset_04nov.png
```

10. Evidências

Evidência	Onde
Logs de console do TX e do RX da referência	`dados/logs/etapa2_tx_*.log`, `dados/logs/etapa2_rx_*.log` (no git)
Registro do ensaio de deriva	`dados/logs/deriva_offset_20261104_104204.json` (no git)
Capturas IQ e figuras da referência	`dados/capturas/ook_ref_20261104_*.{npy,png}` (fora do git pelo tamanho, índice em dados/README.md)
Capturas contínuas (4,1 s e 16 s)	`dados/capturas/ook_ref_continua_20261104.npy`, `dados/capturas/ook_ref_continua16s_20261104_103841.npy` (fora do git)
Diferenças das cópias de ensaio	`ensaios/etapa2/diferencas.diff`
Figuras	`etapa2_04nov.png`, `etapa2_rx_referencia_04nov.png`, `deriva_offset_04nov.png`
Código da correção do A17	`sdrlink/hw/rtl_rx.py` e `tests/test_rtl_rx.py`

11. O que isso muda para a Etapa 3

1. **Leitura**: o receptor do projeto usa só `capturar` (assíncrona). Para os 60 s da prova serão 123 M amostras; o vetor de bytes brutos (246 MB) cabe na memória, e a conversão para complexo será feita em pedaços.
2. **Ganho**: começar em 10 dB lado a lado, conferir `fracao_saturada` antes de cada ensaio e registrar o ganho **aplicado**, não o pedido.
3. **Offset**: estimar por pacote, com busca de pelo menos ±3 kHz (A18).
4. **Todos os quadros**: o decodificador desta etapa já tem a estrutura que falta na referência (A1): achar todos os sincronismos e decodificar cada um. É o ponto de partida do RX multiquadro com número de sequência e CRC-16, e da PER oficial pelo log do TX.

4. Auditoria de conformidade das Etapas 1 e 2

Confronto item por item do enunciado (Projeto_Final_SDR.docx.pdf) com o que foi feito e com a evidência que comprova. Regra: item sem evidência gravada (log, registro JSON, captura ou figura) não conta como atendido.

Legenda: ✅ atendido com evidência, 🟡 parcial, ❌ não atendido, ⏳ depende de terceiros.

1. Entregas das Etapas 1 e 2 (seção 6 do enunciado)

#	Exigência	Situação	Evidência
1.1	Etapa 1: calibrar o RTL-SDR em ppm	✅	+84 ppm com o Pluto da equipe: calibracao_rtl.md seção 10; `dados/calibracao/calibracao_20261104_100631.{json,png}` e `..._100917`; `configs/rtl.toml`
1.2	Etapa 1: medir o offset residual	✅	−356 e −524 Hz logo após a calibração; de −0,68 a −1,62 kHz durante os ensaios; deriva medida (etapa2_recepcao_ook.md seção 6, `dados/logs/deriva_offset_20261104_104204.json`, figura `deriva_offset_04nov.png`)
1.3	Etapa 1: operar as modulações pelo enlace Pluto → RTL	✅	Só a OOK existe nesta fase; 8 de 8 recepções e 698 de 698 quadros (etapa2_recepcao_ook.md)
1.4	Etapa 2: estudar e entender a OOK	✅	00_planejamento_e_auditoria.md seções 4 e 5; achados A1 a A19
1.5	Etapa 2: mapear funções	✅	00_planejamento_e_auditoria.md seção 4.1
1.6	Etapa 2: executar os códigos nos SDRs	✅	`dados/logs/etapa2_tx_*.log` e `etapa2_rx_*.log`; cópias de ensaio com diff em `ensaios/etapa2/diferencas.diff`
1.7	Etapa 2: registrar recepção válida	✅	Log com `CRC-8 received=0x44 computed=0x44 -> OK`, captura IQ e figura do receptor (etapa2_recepcao_ook.md seção 3)
1.8	Toda entrega: registro de uso de IA (ferramentas, prompts, o que foi aproveitado, como foi verificado)	❌	registro_ia.md com a tabela das Etapas 1 e 2 **vazia**. Pendência P1
1.9	Prazo das Etapas 1 e 2: 29/10	❌	Ensaios concluídos em 04/11. Pendência P5

2. Requisitos gerais que já se aplicam (seções 3, 4, 7 e 10)

#	Exigência	Situação	Evidência / observação
2.1	Pluto só como TX, RTL só como RX; receptor do Pluto nunca usado	✅	Nenhum código do projeto abre o RX do Pluto; o ensaio de deriva só lê o sensor de temperatura do Pluto
2.2	TX e RX em computadores distintos	🟡	Ensaios de 04/11 com **um só PC**. Obrigatório nas provas. Pendência P2
2.3	Aquecer o RTL cerca de 10 min antes de calibrar ou ensaiar	✅	RTL ligado antes das 09:57; calibração adotada às 10:06 e 10:09 com o RTL aquecido; ensaios a partir das 10:16
2.4	Correção de ppm calibrada e registrada	✅	item 1.1

#	Exigência	Situação	Evidência / observação
2.5	Ajustar a correção do oscilador do Pluto (recomendado)	🟡	Erro do Pluto da equipe estimado em cerca de −9,5 ppm; `xo_correction` ainda de fábrica. Pendência P3
2.6	Frequência e potência autorizadas (faixa de 915 a 928 MHz, definida pelo professor em 04/11)	🟡	O sinal (915,2 MHz) ficou dentro da faixa, mas o LO do Pluto ficou em 915,000 MHz, na borda, e a imagem IQ em 914,8 MHz caiu fora (A20). Corrigido para os próximos ensaios: `conferir_faixa_autorizada` recusa a transmissão se sinal, LO ou imagem saírem da faixa (com teste)
2.7	Registrar: pacotes transmitidos, recebidos, válidos e perdidos	✅	`decodificar_quadros_ensaio.py`: esperados, detectados, CRC OK, CRC falho, não detectados (seção 4 do registro da Etapa 2)
2.8	Registrar: duração, modulação e parâmetros	✅	Logs e tabela de montagem
2.9	Registrar: ganho do receptor com AGC desligado	✅	Corrigido nesta auditoria: o registro dizia 10 dB, mas o R820T aplica **8,7 dB** (degrau mais próximo). Os documentos passaram a citar o valor aplicado
2.10	Registrar: offset residual	✅	item 1.2
2.11	Registrar logs, parâmetros e capturas IQ para auditoria	✅	Corrigido nesta auditoria: os logs de console não eram gravados. Agora ficam em `dados/logs/`, versionados; capturas IQ em `dados/capturas/` (fora do git pelo tamanho, indexadas em `dados/README.md`)
2.12	Código executável, versionado, documentado e com parâmetros configuráveis	✅	Repositório com tags `bloco-00` a `bloco-02`; portão de qualidade (ruff, mypy --strict, regras NASA, 74 testes); `configs/rtl.toml`
2.13	Toda alteração no código de referência documentada	✅	Originais intactos em `referencia/`; cópias geradas por script com diff versionado

3. Limites das provas (seções 8 e 12), situação hoje

#	Limite	Situação	Observação
3.1	PER ≤ 1%	✅ na OOK de referência	0 / 698 com a nossa captura. Com o leitor da referência seria 14 / 109 (12,8%) por causa do A17. A PER oficial, pelo log do TX, depende do quadro com número de sequência (Etapa 3)
3.2	Banda ocupada a 99% ≤ 1 MHz	⏳	Ainda não medida; obrigatória na Etapa 3
3.3	Goodput ≥ 10 kbit/s (prova B), 500 pacotes	❌ esperado	O próprio enunciado avisa que a OOK de referência não chega lá; é meta da Etapa 3 em diante
3.4	Payload de 128 bytes, número de sequência, CRC adequado	❌	Etapa 3 (achados A3, A4, A9)
3.5	Receptor processa em até 10 min e gera arquivo de saída	❌	Etapa 3 e 5 (A1)

Sobre BER: o enunciado não fixa limite de BER, só manda medir (Etapa 5, com sequência pseudoaleatória). Medido agora, com a mensagem conhecida: 0 erros em 117 264 bits de payload.

4. Problemas encontrados nesta auditoria e o que foi feito

#	Problema	Ação
F1	O registro dizia ganho de 10 dB, mas o rádio aplicou 8,7 dB	Documentos corrigidos; a Etapa 3 registra sempre `Captura.ganho_db` (valor lido do rádio)

#	Problema	Ação
F2	Logs de console das recepções não eram gravados	Ensaio repetido às 10:38 com os logs gravados e versionados (`dados/logs/`)
F3	Offset residual medido só na calibração	Medido em todas as capturas e num ensaio de deriva dedicado: achado A18
F4	A PER tinha sido estimada com só 179 quadros	Captura de 16 s: 698 quadros, PER 0%, BER 0
F5	O topo de `calibracao_rtl.md` ainda mostrava +77 ppm e a figura de 22/10	Topo atualizado para +84 ppm com a figura de 04/11; revisão 2 marcada como histórico
F6	Erro de I2C ao fechar o RTL depois da leitura assíncrona	Investigado e registrado como A19 (não afeta os dados)
F7	LO do Pluto na borda da faixa autorizada, imagem IQ fora (A20)	Faixa conferida no código antes de transmitir; padrão do transmissor de tom passou para 921,25 MHz

5. Pendências

#	Pendência	Responsável	Quando
P1	Preencher o registro de uso de IA das Etapas 1 e 2 (texto próprio da equipe)	Gabriel	antes de entregar
P2	Repetir uma recepção com TX e RX em computadores distintos e gravar o log dos dois lados	equipe	próxima bancada
P3	Decidir sobre o `xo_correction` do Pluto e, se ajustar, recalibrar o RTL	equipe	próxima bancada
P4	Separar a deriva do Pluto da do RTL (rádios FM medidas junto com a OOK)	equipe	Etapa 3
P5	Confirmar com o professor como entregar as Etapas 1 e 2 fora do prazo	Gabriel	já
P6	Frequência oficial: resolvida, o professor definiu 915 a 928 MHz em 04/11. Repetir os ensaios com o LO perto do meio da faixa (A20)	equipe	próxima bancada

5. Anexo A: registro de uso de IA

O professor exige, em cada entrega intermediária, um registro curto: quais ferramentas e prompts foram usados, o que foi aproveitado e, principalmente, **como a resposta foi verificada** (cálculo, simulação ou medição). No final, todos os registros vão consolidados em um anexo.

Etapas 1 e 2 (prazo 29/10)

#	Ferramenta	O que pedi (resumo do prompt)	O que aproveitei	Como verifiquei